

OOP Cheat Sheet (Beginner-Friendly)

1. Class & Object

- Class: Blueprint/template that defines attributes (data) and methods (behavior).
- Object: Actual instance created from a class.
- Analogy: Class = recipe, Object = cake baked from that recipe.

Example:

```
class Car:
```

```
    def __init__(self, make, model):  
        self.make = make  
        self.model = model
```

```
car1 = Car("Toyota", "Camry") # Object
```

```
---
```

2. Encapsulation (Protecting Data)

- Definition: Keep data safe by restricting direct access.
- Why it matters: Prevents accidental misuse, ensures controlled updates.
- Analogy: ATM machine - you don't directly touch the bank's database, you interact through a secure interface.

Example:

```
class BankAccount:
```

```
    def __init__(self, balance):
```

```
self.__balance = balance # private
```

```
def deposit(self, amount):
```

```
    self.__balance += amount
```

```
def get_balance(self):
```

```
    return self.__balance
```

```
---
```

3. Inheritance (Reusing Code)

- Definition: A child class can inherit attributes/methods from a parent class.
- Why it matters: Avoids duplication, organizes code hierarchically.
- Analogy: Family traits - a child inherits features from parents but can add their own.

Example:

```
class Vehicle:
```

```
    def __init__(self, make):
```

```
        self.make = make
```

```
class Car(Vehicle): # Car inherits Vehicle
```

```
    def __init__(self, make, doors):
```

```
        super().__init__(make)
```

```
        self.doors = doors
```

```
---
```

4. Polymorphism (Same Action, Different Forms)

- Definition: Same method name, but different behavior depending on the object.
- Why it matters: Makes code flexible - one function can work with many object types.
- Analogy: "Play" button - plays music in a music app, plays video in a video app.

Example:

```
class Animal:
```

```
    def make_sound(self): pass
```

```
class Dog(Animal):
```

```
    def make_sound(self): return "Bark"
```

```
class Cat(Animal):
```

```
    def make_sound(self): return "Meow"
```

```
def animal_sound(animal):
```

```
    print(animal.make_sound())
```

```
animal_sound(Dog()) # Bark
```

```
animal_sound(Cat()) # Meow
```

```
---
```

5. Abstraction (Hiding Complexity)

- Definition: Show only essential features, hide the internal details.
- Why it matters: Simplifies usage, focuses on what an object does, not how.
- Analogy: Driving a car - you use the steering wheel and pedals, but don't need to know how the

engine works internally.

Example:

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):
```

```
    @abstractmethod
```

```
    def area(self): pass
```

```
class Circle(Shape):
```

```
    def area(self): return 3.14 * 5 * 5
```

```
---
```

Quick Recap

- Encapsulation -> Protect data (ATM analogy).
- Inheritance -> Reuse code (family traits analogy).
- Polymorphism -> Same interface, different behavior (play button analogy).
- Abstraction -> Hide complexity (car driving analogy).